

🧠 STAGE 3 — ANTI-SCRAPING AWARENESS (FULL REVISION NOTES)

🔒 SYSTEM GOAL (LOCK THIS FIRST)

- Jab scraper break ho:
 - **panic nahi**
 - **code turant change nahi**
 - **pehle problem ka naam lagana**
 - Goal = 🗒️ "Why it broke?" samajhna before "How to fix?"
-

1 Detection Awareness — *How sites catch bots*

🔑 Core Idea

- Website **tumhara Python code nahi dekhti**
 - Website sirf **HTTP requests ka behaviour** dekhti hai
-

🧠 Website kya judge karti hai

- Request **speed**
- Request **timing**
- **Regularity / exact pattern**
- **Repetition**
- **Consistency**

🗒️ Agar behaviour **human jaisa nahi**, to bot suspect.

🗣️ Real-life analogy

- Human = ruk-ruk ke kaam karta hai
 - Bot = machine gun jaisa exact speed
-

🔔 Important rules

- Code clean hona $\neq$ safe
 - Selectors perfect hona $\neq$ safe
 - **Behaviour > Code**
-

Problem this solves

- 403 / empty HTML / random failures ka real reason samajh aata hai
 - Galat jagah debugging band hoti hai
-

2 Browser vs Script Traffic

Core Idea

- Website ye dekhti hai:
 - **Browser jaisa traffic** hai ya
 - **Script jaisa naked traffic**
-

Browser traffic (Human)

- Multiple requests jaati hain:
 - HTML
 - CSS
 - JS
 - Images
 - API calls
- Cookies set hoti hain
- Navigation hota hai
- History hoti hai

 Website bole: *"Normal user"*

Script traffic (Bot)

- Ek direct **GET**
- No cookies
- No history
- No context

- No assets

🔗 Website bole: "Suspicious"

🚫 Important clarification

- Browser ko **copy karna goal nahi**
 - Browser ko **study karna goal hai**
-

🧠 Key rule

```
Browser = conversation  
Script = demand
```

3 Request Patterns Matter More Than Code

🔑 Core Idea

- Website **average speed** nahi
 - Website **bursts + rhythm** dekhti hai
-

🔍 Request pattern kya hota hai

- Order (kaun pehle)
 - Timing (gap kitna)
 - Frequency (kitni baar)
 - Consistency (kitna robotic)
 - Context (session grow ho rahi?)
-

⚠️ Dangerous misunderstanding

- "1 sec sleep hai → safe hoon" ❌
 - Burst andar ho to **still unsafe**
-

🧠 Key rule

```
200 OK + empty HTML = pattern issue
```

Async ka truth

- Async = waiting time hide karta hai
- Server ko **bursts dikhti hain**
- Blind async = ban magnet

4 Soft Block vs Hard Block

Hard Block

- Clear rejection
- Examples:
 - 403 Forbidden
 - 429 Too Many Requests
 - Redirect to error / captcha page

 Website chillati hai: "NO"

Soft Block (MOST DANGEROUS)

- Status code = 200 OK
- HTML bahut chhota
- Data missing
- Selectors return **None**

 Website smile karti hai aur **data chhupa leti hai**

Golden Rule

403 / 429 → Hard block
200 + empty HTML → Soft block

Why soft block dangerous

- Code crash nahi hota

- Pipeline quietly corrupt hoti hai
- Tum galat data save kar sakte ho

5 Rate-Based Bans

Core Idea

- "Too many requests" = **per unit time**
- Total count irrelevant hai

Rate-based ban triggers

- Bursts (multiple requests ek saath)
- High concurrency repeat
- Exact timing (clock-aligned)

Async + rate-ban

- Async khud problem nahi
- **Blind async problem hai**

Real rule

Smooth flow > Burst flow

Common misunderstanding

- "Sleep hai phir bhi ban kyu?" → sleep batch ke beech hai, request ke beech nahi

6 Header Fingerprinting

Core Idea

- Website **headers ka combination** dekhti hai
 - Ye combination = **fingerprint**
-

🚫 Beginner mistake

- Browser se headers copy-paste ❌
-

⚠️ Suspicious cases

- Bahut kam headers
 - Bahut perfect headers
 - Har request me same headers
 - Headers aur behaviour ka mismatch
-

🧠 Important rule

```
Missing headers = suspicious  
Perfect headers = suspicious
```

🔍 Browser reality

- Header values mostly stable hoti hain
 - Context change hota hai:
 - cookies
 - referer
 - request order
 - navigation
-

7 Cookies & Sessions

🔑 Core Idea

- Cookies = memory
 - Session = relationship over time
-

❌ Stateless requests

- Har request new
- No cookies

- No continuity

🔗 Website bole: *"Har baar naya banda?"*

☑ Session-based requests

- Cookies reused
 - Same identity
 - Natural flow
-

🗯 Key rule

New session every request = suspicious

🔧 "Works once then fails" reason

- First request probation
 - Next requests:
 - cookies missing
 - session reset → soft block
-

8 Diagnosis Skills — MOST IMPORTANT

🔑 Core Idea

- **Fix later**
 - **Diagnose first**
-

⚙ Diagnosis order (NEVER change)

1. Status code
 2. Response size
 3. Timing pattern
 4. Headers
 5. Cookies / session
-

Mental flow

```
Fail →  
Status →  
Size →  
Pattern →  
Headers →  
Session
```

Common classifications

- 403 / 429 → Hard block
- 200 + small HTML → Soft block
- Bursts → Rate-based
- Static headers → Fingerprint
- No cookies → Session issue

Biggest mistake

- Code change before diagnosis

Scrapers Health Check System

Purpose

- Separate **diagnostic tool**
- Scraper ka part nahi
- Reusable for every site

What it logs

- Status code
 - Response size
 - Time gap
 - Header count + UA
 - Cookies present or not
 - Clear diagnosis message
-

How you use it

- Scraper fails → STOP
 - Run health check on 1–3 URLs
 - Read diagnosis
 - Then redesign scraper
-

STRUCTURE OF SCRAPER HEALTH CHECK SCRIPT

1 Config Section

```
URLS = [  
    "problematic_url_here"  
]
```

2 State

- `requests.Session()`
 - `last_request_time`
-

3 Metrics Collected per Request

- `status_code`
 - `len(response.text)`
 - `time gap`
 - `len(headers)`
 - `cookies present`
-

4 Diagnosis Rules

- `403 / 429` → Hard block
 - `200 + small HTML` → Soft block
 - `gap < threshold` → Rate-based risk
 - `few headers` → Fingerprint risk
 - `no cookies` → Session issue
-

5 Output

- Human-readable diagnosis:

DIAGNOSIS:

- SOFT BLOCK
- RATE-BASED
- SESSION ISSUE

☒ FINAL STAGE-3 OUTCOME

Ab tum:

- panic nahi karte
- blindly fix nahi karte
- problem ka naam lagate ho
- calmly redesign karte ho

👉 **This is professional scraping mindset.**
